

CAMASIM: A Brief Report

Dean Hill

January 5, 2026

Abstract

CAMASIM aims to simulate the formation of celestial discs from clouds of particles through repeated collisions over a long period of time. CAMASIM utilises octree-based collision handling, and gravity algorithms such as the Barnes-Hut algorithm. CAMASIM simulates particle interactions, samples and records frames, then visualises the stored frames with a separate program using the Visualisation Toolkit (VTK) library. The simulation also utilises CPU parallelism via OpenMP and GPU acceleration via CUDA to optimise performance.

The project is still in development.

1 Introduction

Celestial discs are common structures within the universe with examples such as planetary rings and many galaxies.

The Celestial-Angular-Momentum-Averaging-Simulation (CAMASIM) project aims to approximately test and simulate how celestial discs, in particular those of planetary systems. The particular method of formation being tested is formation by massively repeated collisions between orbiting bodies, during which small amounts of energy are transformed into non-mechanical forms and the motion of the two bodies becomes more closely aligned. This continues until a large amount of bodies remaining bound to the system have motions so closely aligned about the central body that a disc is formed.

CAMASIM's genesis is in a piece of short-form physics content that I have been unable to find again covering the topic of celestial discs. I deemed the problem an interesting one to implement and an opportunity to further develop my HPC skills.

CAMASIM's primary goal is to approximately test this process by deterministically simulating inelastic particle collisions and gravity for a large number of bodies surrounding a central body as well as being able to efficiently visualise the process.

It should be noted that CAMASIM is not yet complete as discussed in section 6.

2 Algorithmic Model

Celestial discs usually contain billions to multiple orders of magnitude more particles, CAMASIM must be able to scale efficiently with increasing particle count. The most significant barriers to scalability are simulating gravity and particle collisions.

2.1 Octrees for Gravity and Collision Detection

The brute-force method for detecting particle collisions, that is conducting all-pairs comparison, has time complexity $\mathcal{O}(n^2)$, where n is the number of particles, severely limiting scalability especially in the case of dense particle clouds.

Additionally, the direct-sum algorithm for simulating N-body gravity, that is again an all-pairs algorithm, while being the most accurate has time complexity $\mathcal{O}(n^2)$ making it similarly unsuited for larger values of n as desired by CAMASIM.

A solution to both of these is spatial subdivision, in particular the use of octrees, a tree data structure that recursively divides three-dimensional space into small regions referred to as octants. Octrees are particularly applicable here as opposed to uniform spatial divisions such as defining chunks, for its use in the Barnes-Hut algorithm, an $\mathcal{O}(n \times \log n)$ method for N-body approximations, i.e. gravity. The Barnes-Hut algorithm relies on approximating distant collections of particles as singular "super-particles", utilising internal nodes of the octree to store the approximate centre of mass of all particles contained which is calculated using recursive methods.

Furthermore, the octree structure allows for spatial pruning when considering what particles a given particle can collide with, by excluding non-intersecting octants and their particles. By leveraging the octree structure to exclude regions of space, the collision detection algorithm is reduced to time complexity $\mathcal{O}(d \times n)$, where d is the maximum number of particles allowed within each octant (used to prevent possibly infinite recursive subdivision).

The ability to utilise the same octree for both algorithms is highly alluring. Additionally, octrees are a prime candidate for CPU parallelism (achieved using OpenMP) as each child octant can be computed independently of the other during generation and applying the algorithms.

Note that some octree implementations can utilise cache-behaviour and locality incredibly well, I had erroneously not considered researching these algorithms before designing the core structure, resulting in a near entirely random read and write behaviour for the above algorithms. This is discussed further in section 6.

2.2 GPU Acceleration

CAMASIM utilises GPU acceleration via CUDA to massively increase performance in tasks such as applying forces, acceleration, and velocity for each timestep. For larger numbers of particles, the benefits of GPU acceleration becomes increasingly pronounced over CPU alternatives.

2.3 Deterministic Behaviour

One goal of CAMASIM is to have highly deterministic behaviour for a given set of starting parameters. CAMASIM uses seed-based random engines to generate the initial scene such as particle positions, velocities, radii, masses, etc. Unfortunately this has to be done serially to ensure maximum determinism, however this may be unpredictably

affected by compiler optimisations. The algorithms used are also perfectly deterministic, barring possible floating-point arithmetic quirks.

3 Visualisation

Rendering massive numbers of particles can be computationally demanding, to achieve a desirable combination of performance and development speed, I decided to use the Visualisation Toolkit (VTK). CAMASIM does not render frames in real-time, instead the visualisation process is broken down into recording desired frames, then visualising them with a separate program.

The simulation program uses a `Recorder` class to periodically sample and record frames in a custom binary format. After the simulation finishes, the binary is interpreted to a `.pvd` collection for compatibility with VTK.

To render the particle cloud, the `vtkPointGaussianMapper` was used which uses Gaussian splat "sphere-imposters" to efficiently render a massive number of points.

4 Validation

As CAMASIM does not yet have full functionality, extensive testing and validation has not yet been performed, however from observation the simulation does currently behave as expected.

5 Performance

Performance, and maximising the reasonable simulation length/size is a prevalent design factor for CAMASIM. For future reference, all tests were performed on an Acer Nitro 5 laptop, equipped with 16 GB of RAM, a 10th generation Intel i5 CPU (8 cores), and a NVIDIA GeForce GTX 1650 Ti Mobile (1024 cores).

As of currently, the simulation is compute-bound by collision detection and handling. A 100,000 frame simulation of 100,000 particles takes approximately 1 h with an average CPU usage of approximately 97.5 % across every core and a consistent memory usage of approximately 224 MB.

The octree generation is the next limiting component. A 100,000 frame simulation of 10 million particles with collision handling disabled took approximately 1 h, again nearly maximising CPU usage across all cores.

The least limiting component is the application of acceleration and velocity (force application has not yet been needed) where GPU acceleration has been used. I have yet to do a comparable benchmark, however without octree generation and collision handling, the simulation computed approximately 300,000 frames of 1 billion particles in less than 10 min.

The visualisation program is dominated by the process of reading the `.pvd` collection and interpreting as opposed to the actual rendering, even in the billions of particles.

Note that these benchmarks were rather informal and without proper documentation. Extensive benchmarking will be done once all functionality is implemented.

6 Limitations and Future Work

CAMASIM as of currently is still in development with the Barnes-Hut algorithm and a more accurate collision handling algorithm still needing to be implemented. As well as improved usage and benchmarking.

Additionally, a frequent feature seen within the codebase is the use of `PrecT` templating to denote the floating-point type to be used. As design goals shifted this became a massive hindrance and significantly increased compile times. In the future it'd be preferential to use a pre-processor macro to define the precision type.

The original data structure used in CAMASIM to store particle information (structure of arrays) eventually became a notable problem as I implemented octree-based algorithms which rather than manipulating particle information directly instead manipulated particle indexes. The manipulation of particle indexes lead to a large amount of scattered reads and writes, possibly leading to the collision handling bottle-neck. While this will likely not be overhauled, it is a good learning point for the feature.

7 Conclusion

While working on CAMASIM I have learnt an incredible amount such as VTK, OpenMP, CMake, MPI, and more in-depth CUDA techniques (e.g. grid-stride patterns). Additionally, I have been exposed to HPC significantly more and have many takeaways from this project.

Given CAMASIM is still in development I cannot yet properly evaluate it in terms of the original goals.